Data structure and Programming II

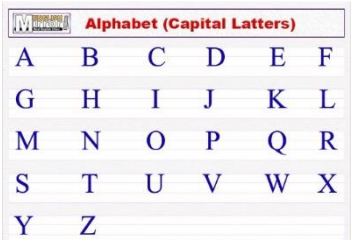
TP05: Linked list (part2)

- Objective:
  - To practice create linked list advanced data structure that is able to store unlimited data
  - Implement add to end, add to begin functions and delete from begin function
  - Apply C++ programming
- Individual work
- Submit to Moodle (do not zip)
  - *A pdf report (cover, exercise and solution with screenshot)*
  - *Source codes*

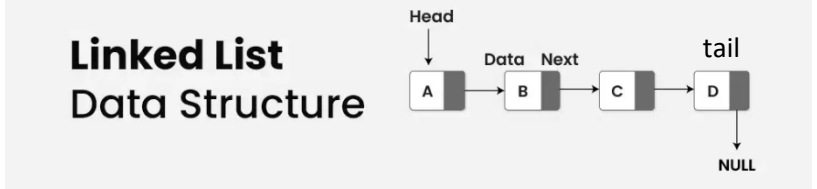
Deadline:
6 days

Write C++ programs to create and implement linked list data structure below.

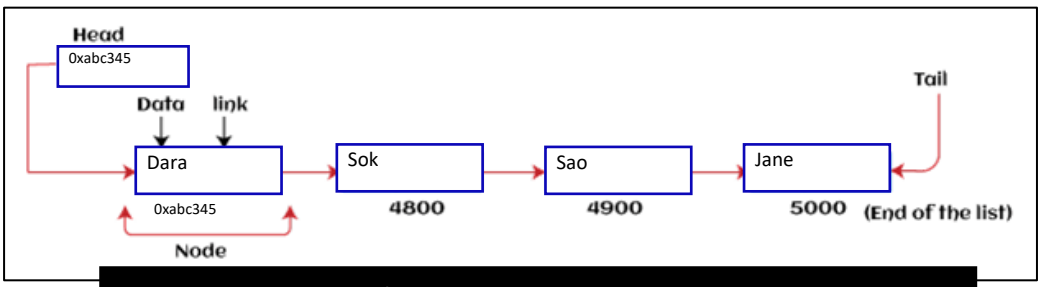
1. Create two linked list that are able to store list of English alphabets, one list for the small letters and another list for the capital letters. **Hint**: Create 2 structures such as element structure and list structure. Then create 3 functions: createList, addPerson(add to the end of the list), display all data in the list.



An example of linked list storing the 4 English capital letters A-D



2. Store names of 50 person in a linked list. Make sure we can add person more as many as possible. **Hint**: Create 2 structures such as element structure and list structure. Then create 3 functions: createList (make the list empty), addPerson(add new person name to the begin of the list), display all data in the list. Finally, test your program in main and add 10 of your friends' names.



An example of a linked list containing 4 person names

3. Design a program that is able to manage product information. Define a linked list that is able to store a list of products. Each product has some , such as productid, Name, price. Create data structures for this program. Create some functions below:
   a. create an empty list,
   b. add product to begin of the list (when the price is less than 50 dollars),
   c. add product to end of the list (when the price is at least 50 dollars), and
   d. display product for the implementation of linked list.

Finally, test your program in main.

```
struct   Product {
        ....
        ....
        ....
};
```

```
struct   ListProduct {
        ....
        ....
        ....
};
```

```
List *createEmptyList(){
        ....
        ....
        ....
}
```

```
void display(.......){
        ....
        ....
        ....
}
```

```
void addProductToBegin(.......){
        ....
        ....
        ....
}
```

```
void addProductToEnd(.......){
        ....
        ....
        ....
}
```

4. With the extension to the exercise #3, we also create two search functions and one update function. The update function is to update data based on the input of productid. The first search function is able to search and display detail product information by name, while the second search function should be able to show all products based on a given price (**p**). It shows product into two sections such as:
   1) Section1: All products that have price lower than p, and
   2) Section2: All products that have price more than p. Test your program in main.

```
void searchByName(.......){
        ....
        ....
        ....
}
```

```
void searchByPrice(.......){
        ....
        ....
        ....
}
```

```
void updateByProductId(.......){
        ....
        ....
        ....
}
```